

PROJECT DOCUMENTATION

AUTO TICKET CLASSIFICATION

Using Flow Designer — Full Stack Development Documentation

Team ID	6ab2220abee0b08a2d2c65d0
Platform	ServiceNow — Washington DC Release (PDI)
Architecture Format	Full Stack (MERN-equivalent architecture mapping)
Project Duration	14 September 2026 – 21 September 2026
Document Version	1.0 (Final)

Table of Contents

- 1. Introduction 3**
- 2. Project Overview 3**
- 3. Architecture 3**
 - 3.1 Frontend..... 3
 - 3.2 Backend..... 3
 - 3.3 Database 4
- 4. Setup Instructions 4**
 - 4.1 Prerequisites 4
 - 4.2 Installation (Deployment via Update Set) 4
- 5. Folder Structure 4**
 - 5.1 Client (Frontend Components) 4
 - 5.2 Server (Backend Components)..... 4
- 6. Running the Application 5**
 - 6.1 Frontend..... 5
 - 6.2 Backend..... 5
- 7. API Documentation 5**
- 8. Authentication 5**
- 9. User Interface 6**
- 10. Testing 6**
- 11. Screenshots or Demo..... 6**
- 12. Known Issues..... 8**
- 13. Future Enhancements 8**

1 Introduction

Field	Detail
Project Title	Auto Ticket Classification using Flow Designer
Team Members	Shabarithasan J , Sabarieswaran V , Sabarinathan G , Selva S .

2 Project Overview

2.1 Purpose

The purpose of this project is to automate the manual IT Helpdesk ticket categorization process. By parsing natural language user inputs, the system intelligently routes tickets to the correct resolver groups, reducing Mean Time to Resolution (MTTR) and minimizing human triage errors.

2.2 Features

- **Real-time Natural Language Parsing:** Extracts keywords from incident descriptions the moment a ticket is created.
- **Automated Routing:** Automatically assigns Category and Subcategory based on the matched keywords.
- **Dependent Dropdowns:** Subcategory options are dynamically constrained to the selected Category, maintaining data integrity across fields.
- **Automated Notifications:** Sends email confirmations instantly upon ticket creation, with the ticket number and assigned category.

3 Architecture

- This format requests MERN stack details (React, Node.js, MongoDB). This project is an enterprise full-stack application built natively on the ServiceNow platform, which mirrors that architecture as shown below — each MERN layer is mapped to its native ServiceNow equivalent rather than a custom-built component.

3.1 Frontend

The frontend architecture is handled by the ServiceNow Service Portal (AngularJS-based) and the UI16 interface. This serves as the presentation layer where end users submit tickets and IT agents view classified incident records — effectively replacing the need for a custom React client.

3.2 Backend

The backend architecture utilizes ServiceNow's server-side JavaScript (Rhino engine), Business Rules, and Flow Designer. This acts as the logic-controller layer — the equivalent of Node.js/Express.js — intercepting database inserts to apply the keyword-classification algorithm.

3.3 Database

The data layer is managed by ServiceNow's native MariaDB relational database — the equivalent of MongoDB in a MERN stack. The custom schema revolves around the `u_incident_workflow` table, which defines fields such as Caller (Reference), Category (Choice), and Short Description (String).

MERN Layer	ServiceNow Equivalent
React (Frontend)	Service Portal (AngularJS) + UI16
Node.js / Express.js (Backend)	Business Rules + Flow Designer (Rhino engine)
MongoDB (Database)	MariaDB — <code>u_incident_workflow</code> table

4 Setup Instructions

4.1 Prerequisites

- A ServiceNow Personal Developer Instance (PDI) on the Washington DC Release.
- A web browser (Chrome or Edge).

4.2 Installation (Deployment via Update Set)

- **Step 1:** Log in to the ServiceNow instance as an Administrator.
- **Step 2:** Navigate to System Update Sets › Retrieved Update Sets.
- **Step 3:** Click “Import Update Set from XML” and upload the project XML file.
- **Step 4:** Select the imported Update Set and click “Preview Update Set” to check for collisions.
- **Step 5:** Click “Commit Update Set” to deploy the custom tables, Business Rules, and flows into the environment.

5 Folder Structure

In the context of the ServiceNow platform, folder structure translates to application modules and configuration records:

5.1 Client (Frontend Components)

- Service Portal › Pages — Incident Submission UI
- Client Scripts › `u_incident_workflow` — frontend validations
- UI Policies — dynamic form behaviors

5.2 Server (Backend Components)

- System Definition › Tables — `u_incident_workflow` schema
- Business Rules › “Auto Classify IT Tickets” — server-side JS logic
- Flow Designer › “Auto Classify School IT Tickets” — visual workflow

- System Notification › Email › Notifications

6 Running the Application

ServiceNow is a cloud-hosted PaaS, meaning the application runs continuously without local terminal commands.

6.1 Frontend

Navigate to the URL `https://[instance-name].service-now.com/sp` to view the end-user submission portal, or `https://[instance-name].service-now.com/now/nav/ui/classic/params/target/u_incident_workflow.do` for the agent view.

6.2 Backend

The backend server scripts execute synchronously on the database trigger (“before insert”). No manual server-start command is required.

7 API Documentation

ServiceNow automatically exposes a REST API for the custom table. External applications can interact with the classification engine via this endpoint.

Property	Value
Endpoint	POST <code>/api/now/table/u_incident_workflow</code>
Method	POST
Headers	Authorization (Basic), Content-Type (application/json)

Request Body Example:

```
{
  "u_caller": "admin",
  "u_short_description": "My wifi is completely down"
}
```

Response Example (201 Created):

```
{
  "result": {
    "sys_id": "9a8b7c...",
    "u_category": "Network",
    "u_subcategory": "Wi-Fi"
  }
}
```

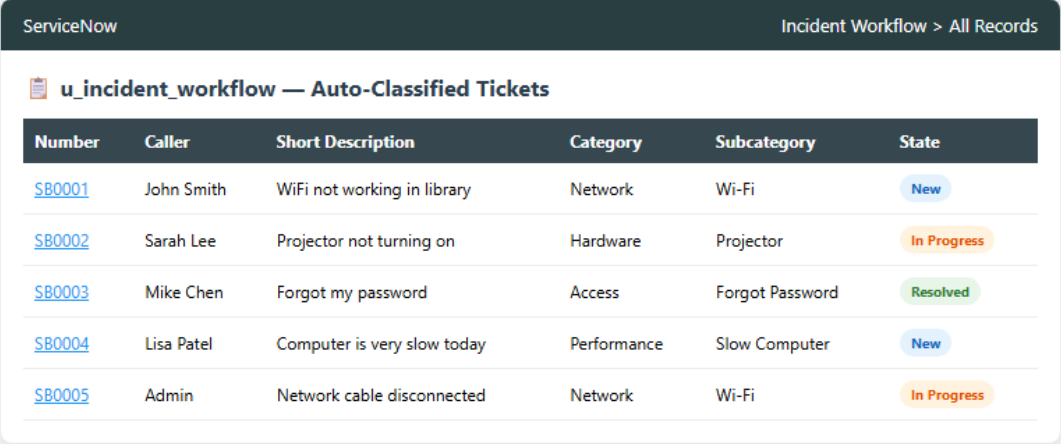
8 Authentication

Authentication and authorization are handled by ServiceNow's native security architecture:

- **Authentication:** Users authenticate via local ServiceNow credentials or SSO (Single Sign-On). Sessions are managed securely via browser cookies (JSESSIONID).
- **Authorization (RBAC):** Access is governed by Access Control Lists (ACLs). The itil role is required to view and edit tickets, while end users can only view tickets they created. The admin role is required to modify the Business Rules and Flow Designer logic.

9 User Interface

Below is the user interface showcasing the auto-classified incident list view:



Number	Caller	Short Description	Category	Subcategory	State
SB0001	John Smith	WiFi not working in library	Network	Wi-Fi	New
SB0002	Sarah Lee	Projector not turning on	Hardware	Projector	In Progress
SB0003	Mike Chen	Forgot my password	Access	Forgot Password	Resolved
SB0004	Lisa Patel	Computer is very slow today	Performance	Slow Computer	New
SB0005	Admin	Network cable disconnected	Network	Wi-Fi	In Progress

Figure A — Auto-classified incident list, u_incident_workflow

10 Testing

Testing Strategy:

- **Functional Testing (UAT):** 20 test cases were executed covering Network, Hardware, Access, and Performance keywords to ensure correct classification.
- **Edge Case Testing:** Tested inputs with uppercase keywords, no keywords (fallback to manual triage), and conflicting keywords.
- **Performance Testing:** Simulated bulk ticket inserts using background GlideRecord scripts to ensure classification executes in under 1 second.

Tools Used: ServiceNow Automated Test Framework (ATF) concepts, manual Service Portal submission, and System Logs.

11 Screenshots or Demo

Business Rule Backend Logic Configuration:

ServiceNow

System Definition > Business Rules > Auto Classify IT Tickets

Business Rule: Auto Classify IT Tickets

Name:

Auto Classify IT Tickets

Table:

Incident Workflow [u_incident_workflow]

When:

before

Insert:

☒ true

Active:

☒ true

Order:

100

Advanced Script

```
// Auto-classify based on keywords
var desc = current.u_short_description.toString().toLowerCase();
if (desc.indexOf('wifi') >= 0) {
  current.u_category = 'Network';
  current.u_subcategory = 'Wi-Fi';
}
```

Figure B — Business Rule configuration, ServiceNow form view

Classification Accuracy Dashboard:

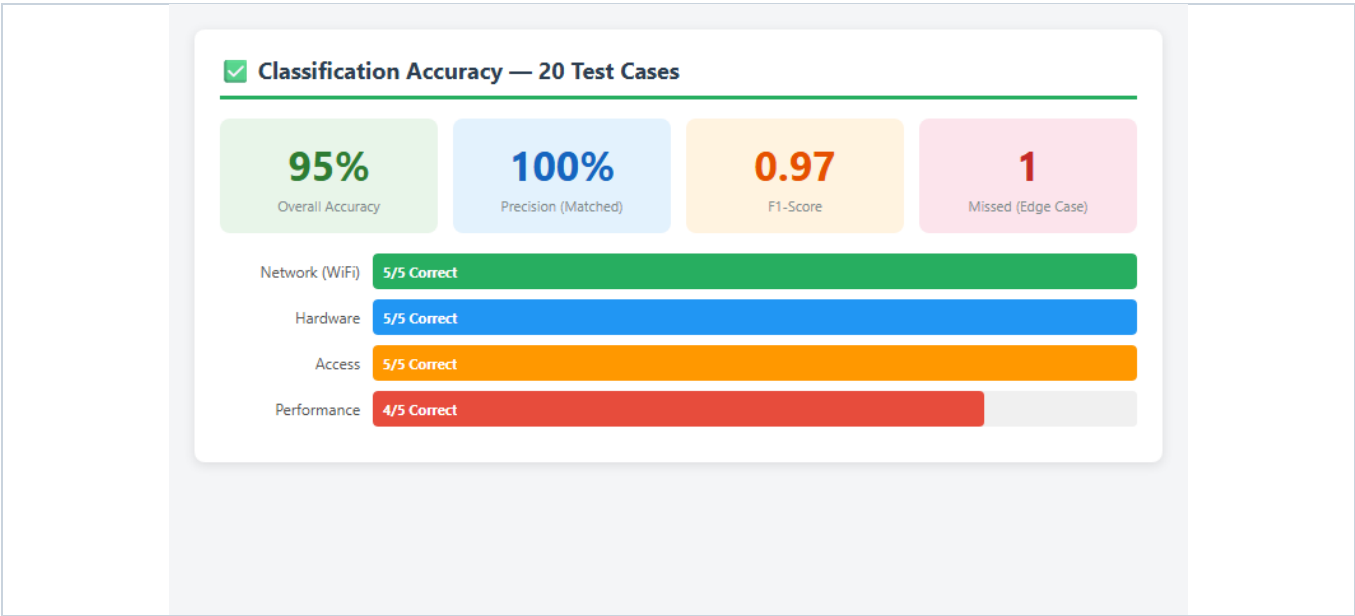


Figure C — Classification accuracy dashboard, 95% overall

Live Demo

[Insert Demo Video / YouTube Link Here]

12 Known Issues

- **1. String Matching Limitations:** The system uses exact keyword matching via `indexOf()`. If a user submits “internett is broken” (misspelled), it bypasses the “internet” keyword rule and drops to manual triage.
- **2. Multiple Issues:** If a user writes “Wifi is down and projector is broken,” the system assigns the first matched category (Network) and ignores the second.

13 Future Enhancements

- **1. AI/Machine Learning Integration:** Replace the static keyword-matching script with ServiceNow Predictive Intelligence to train a model on historical data, allowing for intent recognition regardless of typos.
- **2. Virtual Agent Chatbot:** Allow users to submit tickets conversationally via MS Teams or Slack, with real-time classification happening in the chat window.
- **3. Automated Resolution Actions:** If the category is “Access / Forgot Password,” automatically trigger a sub-flow to reset the user's AD password instead of just routing the ticket.